



+



Hands-on workshop

Build it. Break it. Prove it.

Scaffolding, debugging and testing a real app with GitHub Copilot agent mode

Contoso Outfitters storefront

React · Node · Java

48 minutes

AGENDA

One app, three modules, 48 minutes

0:00 · 3 MIN

Kickoff

Tour the app, the repo instructions, prompt files and agents

0:03 · 13 MIN

Module 6

Scaffolding. A feature brief becomes working code across API, service and UI

0:16 · 16 MIN

Module 7

Debugging and RCA. Logs, request IDs and the contract lead to root causes

0:32 · 14 MIN

Module 8

Testing. TDD, BDD and spec-driven development, tests first

0:46 · 2 MIN

Wrap-up

Three habits to take back to your team

MEET THE APP

Contoso Outfitters is a real storefront that's still in development

localhost:5173

Contoso Outfitters

Shop

Orders

Inventory

API: node

Products

footwear

Trail Running Shoes

SKU-1001

\$89.99

Add to cart

apparel

Merino Wool Socks (3-pack)

SKU-1002

\$24.50

Add to cart

gear

Insulated Water Bottle

SKU-1003

\$32.00

Add to cart

apparel

Ultralight Rain Jacket

SKU-1004

\$149.00

Add to cart

gear

Headlamp 400 Lumen

SKU-1005

\$44.95

Add to cart

gift

Gift Card

SKU-1006

—

Add to cart

Cart

Customer

Guest checkout

Your cart is empty.

Promo code

Apply

Place order

client/ React 18 + Vite

server-node/ Express 5

server-java/ Spring Boot 3

One contract: docs/API.md

GitHub Copilot workshop

03

Copilot works inside **your repo's rules**, not a toy snippet

`copilot-instructions.md`

- **Parity rule:** change one backend, change both and update the contract
- Structured logs with a `requestId`
- Error envelope, no stack traces, no PII in logs

`prompts/`

- Reusable slash commands
- `/scaffold-feature /rca`
- `/fix-and-prove /tdd-next`
- `/bdd-steps /sdd-plan`

`agents/`

- **debugger:** root cause, failing test, then fix
- **test-writer:** tests only, never changes production code without asking
- Per-stack `*.instructions.md` apply by path

MODULE 6 · 13 MINUTES

Scaffolding

From a feature brief to working, tested code across API, service and UI, following the patterns already in the repo.

Inline completion

Agent mode

Prompt files

The comment **is** the prompt

 server-node/src/routes/catalog.js

```
export const catalogRoutes = ({ store }) => {  
  const router = Router();  
  router.get('/products', ...);  
  // GET /products/:sku returns one product, or throws  
  // NotFoundError if the SKU doesn't exist  
  router.get('/products/:sku', (req, res) => {  
    const product = store.findProduct(req.params.sku);  
    if (!product) throw new NotFoundError(...);  
    res.json(product);  
  });  
  return router;  
};
```

What to point out

It reaches for `NotFoundError` from `src/errors.js` without being told. The style comes from the file around it.

Java version

`CatalogController.java`, under `products()`:

```
/** Returns one product by SKU,  
or throws NotFoundException if it  
doesn't exist. */
```

Every customer has loyalty points, but nothing reads them

As a returning customer, I want to earn points when I place an order and see and redeem my balance at checkout, so I have a reason to come back.

Ada Lovelace · C-100

120 points in seed.json

Out of scope: point expiry, discounts from points, tiers and bonuses. Module 8 adds those test-first.

THE BRIEF ASKS FOR	DETAIL
GET <code>/customers/{id}/loyalty</code>	Balance, or 404
POST <code>.../loyalty/redeem</code>	400 · 404 · 409 INSUFFICIENT_POINTS
Earn on checkout	1 point per whole dollar, in its own calculator
Loyalty panel	Under the cart, hidden for guests
Two log lines	earned and redeemed, no PII

One prompt file scaffolds the feature, but **plans before it codes**

 Copilot Chat

Agent

```
/scaffold-feature spec=specs/loyalty-  
points/FEATURE.md stack=node
```

WITHOUT THE PROMPT FILE

```
#codebase Implement specs/loyalty-points/FEATURE.md for  
server-node and the client. Follow the patterns in the  
pricing and orders features. Show me a file plan first.
```

- 1 Reads the **spec** , **docs/API.md** and the instructions
- 2 Finds the **closest feature** to mirror
- 3 Shows a **file plan** and stops
- 4 You review it and reply **Approved**
- 5 Builds service, route, UI, contract
- 6 Writes tests and **runs the suite**

Expect 6 to 9 files that copy the pricing feature's shape

LAYER	NODE	JAVA
Service	loyaltyCalculator.js, loyaltyService.js	loyalty/ package: Calculator, Service, records
API	routes/loyalty.js, edits to app.js, orderService.js, errors.js	LoyaltyController, InsufficientPointsException, edits to Order, OrderService
Client	api/client.js, components/LoyaltyPanel.jsx, App.jsx	
Tests	test/loyalty.test.js	LoyaltyControllerTest

✓ Constructor injection

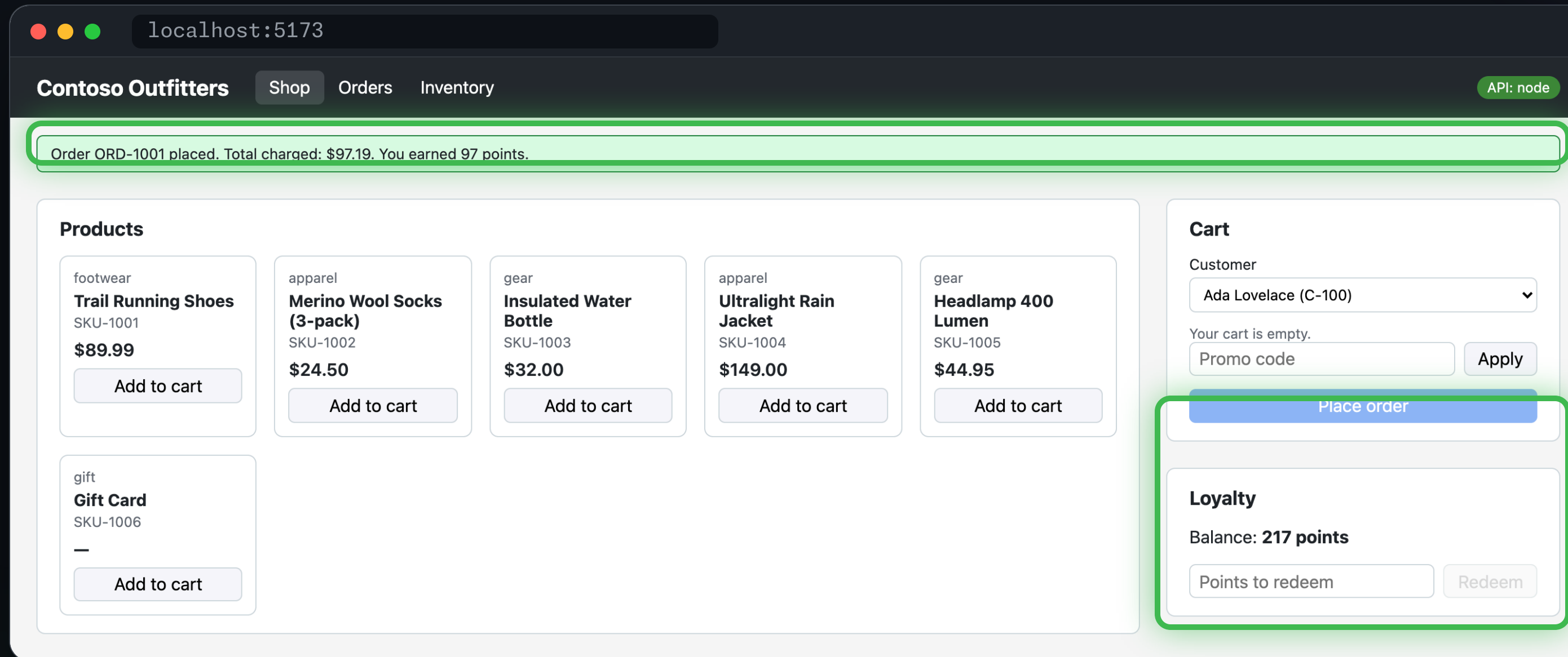
✓ Existing error hierarchy

✓ Structured logger

✓ docs/API.md updated



Ada buys trail shoes for \$97.19 and earns 97 points



120 points to start

+97 for a \$97.19 order

217 points now

Then redeem 50

A prompt file turns a good prompt into a **team workflow**

`.github/prompts/scaffold-feature.prompt.md`

name: scaffold-feature

agent: agent

argument-hint: "spec=... stack=node|java|both"

Scaffold the feature in `${input:spec}`
for backend `${input:stack}`.

Before you write any code:

1. Read the spec, docs/API.md, instructions
2. Find the closest existing feature
3. **Show me a file plan and wait for my OK**

agent: agent

Runs in Agent mode, so it can edit files and run tests

`${input:...}`

One prompt serves any spec and either stack

Plan → approve → build → test

The structure is what makes the output predictable

MODULE 7 · 16 MINUTES

Debugging and RCA

Use real evidence (stack traces, logs, terminal output and the API contract) to find root causes, then prove each fix with a test.

Runtime exception

Silent logic defect

Data integrity

Build error

Five symptoms, and each needs a different prompt

DO THIS	YOU SEE	DEFECT TYPE	PROMPTING STRATEGY
Add Gift Card to cart	500 + request ID	Runtime exception	Request ID + logs, trace before fixing
Rain Jacket + SAVE10	Tax 11.92, total 146.02	Silent logic	Expected vs actual + the contract
Apply SAVE99	Node: accepted, -\$0.00	Contract violation	Ask why, compare both stacks
Inventory → Reconcile	500 + request ID	Data integrity	Debugger agent, check the numbers
Demo: break the build	Won't start or compile	Build error	#terminalLastCommand: explain why

Adding a Gift Card to the cart **returns a 500**

localhost:5173 · Cart

Cart

Customer

Guest checkout

Gift Card

− 1 +

Promo code

Apply

INTERNAL_ERROR An unexpected error occurred
request 1a6f5cc8

Place order

⚠ The symptom

The quote fails with **INTERNAL_ERROR**. No stack trace leaks to the browser, which is correct.

💡 The thread to pull

The banner shows `request 1a6f5cc8`. Every log line for this request carries that ID in `logs/app.log`.

Node: 500**Java: 500**

Give Copilot the request ID and the log, then ask for the root cause

 Copilot Chat

Ask

```
/rca Checkout with a Gift Card in the cart returns  
500. Request ID 1a6f5cc8. #file:logs/app.log
```

 logs/app.log

```
ERROR [1a6f5cc8] ErrorHandler - Unhandled error on POST  
/api/cart/quote  
TypeError: Cannot read properties of null (reading 'amount')  
    at #priceLine (src/services/pricingService.js:66:37)
```

Root cause

SKU-1006 has "price": null in the seed data, and pricing assumes every product has a price.

Blast radius

Java fails too:
NullPointerException in
PricingService.priceLine.

The better question

Fix the cause, **prove it with a failing test first**

 Copilot Chat

@debugger

Fix the Gift Card checkout crash using the option you recommended. Write the failing test first.

✓ Good fix

Validate and return **400**
VALIDATION_ERROR with a
clear message, in both
backends

✗ Bad fixes

Price it at 0, or wrap it in
`try/catch`

After the fix

Cart

Customer

Guest checkout ▼

Gift Card - 1 +

Promo code Apply

VALIDATION_ERROR Product SKU-1006 has no price and cannot be purchased

Place order

The rain jacket quote is wrong, and **nothing errors**

Rain Jacket + SAVE10

Ultralight Rain Jacket − 1 +

SAVE10

Apply

Subtotal	\$149.00
Promo SAVE10 ✓	−\$14.90
Tax (8%)	\$11.92
Total	\$146.02

Place order

WHAT THE APP DOES

$$149.00 \times 0.08 = 11.92$$

WHAT THE CONTRACT SAYS

$$(149.00 - 14.90) \times 0.08 = 10.73$$

"Tax is charged on the **discounted** amount." docs/API.md

Give it the numbers and the contract, and ask it to explain first

Copilot Chat

Ask

```
#file:docs/API.md #file:logs/app.log
```

For SKU-1004 x1 with promo SAVE10, the quote shows subtotal 149.00, discount 14.90, tax 11.92 and total 146.02. Using the pricing rules in the API contract, what should tax and total be? Find the line of code responsible. Explain before you fix.

```
> The evidence in the log
```

```
quote computed subtotal=149 promo=SAVE10 discount=14.9 taxRate=0.08
tax=11.92 total=146.02
```

After /fix-and-prove

Ultralight Rain Jacket - 1 +

SAVE10

Apply

Subtotal	\$149.00
Promo SAVE10 ✓	-\$14.90
Tax (8%)	\$10.73
Total	\$144.83

Place order

/fix-and-prove

10.73 · 144.83

An unknown promo code is **silently accepted** on Node

Before

SAVE99	Apply
Subtotal	\$149.00
Promo SAVE99 ✓	-\$0.00
Tax (8%)	\$11.92
Total	\$160.92

After

SAVE99	Apply
INVALID_PROMO Promo code SAVE99 is not valid	

Copilot Chat

Ask

Why was SAVE99 accepted? The contract says unknown codes are 400 INVALID_PROMO.

The root cause

```
const promo = promos.get(code); // undefined
if (promo === null) throw ...    // never true
... promo?.percent ...          // hides it
```

Parity talking point: Java uses `Optional.orElseThrow()` and never had this bug.

Reconcile crashes on a SKU **nobody ever registered**

localhost:5173 · Inventory

Reconcile stock count

One line per SKU: SKU, counted

SKU-1001,40
SKU-1002,250
SKU-1003,130
SKU-1004,21
SKU-1005,64
SKU-99871,12

Reconcile

INTERNAL_ERROR An unexpected error occurred
request cca5e65e

logs/app.log

```
TypeError: Cannot read properties of undefined  
(reading 'onHand')  
    at InventoryService.reconcile  
(inventoryService.js:39:50)
```

Copilot Chat

@debugger

Reconciling the warehouse count sheet in the Inventory tab returns 500. Find the request in [logs/app.log](#), explain the root cause, and fix it so it meets the contract in [docs/API.md](#) (unknown SKUs are skipped and reported, and averageVariance is the mean absolute variance).

Java: NullPointerException ... because "expected" is null, an unboxing crash on the same SKU.

The crash fix isn't the whole fix: the numbers are still wrong

AFTER "SKIP UNKNOWN"	NODE	JAVA
Variance rows	2	4 (SKU-1002, 1003 show 0)
Average	0.5	0.25
Remaining bug	Signed average: $(-2 + 3) / 2$	Signed average and Integer <code>!= Integer</code> above 127

Correct result

Reconcile			
SKU	Expected	Counted	Variance
SKU-1001	42	40	-2
SKU-1004	18	21	3

Average variance: 2.5

Unregistered SKUs skipped: SKU-99871

avg 2.5

unknownSkus: SKU-99871

Java talking point

250 equals 250, but two boxed 250s are different objects. The JVM only caches -128 to 127, so tests with small numbers never catch it. Ask for a test with stock above 127.

A teammate pushed a half-finished rename

```
> Node · npm start
```

```
SyntaxError: The requested module './services/pricingService.js'  
does not provide an export named 'PricingService'
```

```
> Java · mvn -q compile
```

```
cannot find symbol: method quote(QuoteRequest)  
in CartController and OrderService
```

Tip: run it in a regular terminal, not a task terminal, so `#terminalLastCommand` can see it.

 Copilot Chat

Ask

```
#terminalLastCommand Explain this  
failure. What was the refactor trying  
to do, and what's the minimal change  
that completes it without changing  
the public contract?
```

- A** Revert the rename
- B** Finish it: update every caller
- ?** Ask: which keeps the public API stable?

MODULE 8 · 14 MINUTES

Testing

Use Copilot to write tests first and let them drive the implementation.

TDD

BDD

Spec-driven

/doc

Pick the exercises that fit your audience

A · TDD

Loyalty calculator v2 ·
/tdd-next · 5 min

B · BDD

Discount eligibility ·
/bdd-steps · 4 min

C · SDD

Password reset · /sdd-
plan · 4–8 min

D · Docs

Anything you touched ·
/doc · 1–2 min

IF THE AUDIENCE IS...	RUN
Developers	A (TDD) + D (/doc) + C (plan only)
QA or BA heavy	B (BDD) + C (SDD)
Mixed (default)	A (two loops) + B

One requirement, one failing test, **then stop**

 Copilot Chat

Agent

```
/tdd-next requirements=specs/loyalty-  
points/REQUIREMENTS.md stack=node
```

Red

Write one failing
test

Green

Minimum code to
pass

Refactor

Then stop and
wait

LOOP 1 · REQUIREMENT 2

10% bonus at \$100 or more. The test **\$150** → **165** fails because v1 returns 150.

LOOP 2 · REQUIREMENT 3

InvalidPurchaseError for amounts ≤ 0
(InvalidPurchaseException in Java).

Watch for: checkout must not call the
calculator with a \$0 total.

One .feature file is the source of truth for both stacks

specs/discount-eligibility/discount-eligibility.feature

Background:

Given the store has the following discount tiers:

	tier	minimum_items	minimum_subtotal	discount_percent
	none	0	0	0
	bulk	10	0	5
	vip	0	500	15

Scenario: Cart qualifies for both bulk and VIP thresholds

Given a cart with 15 items and a subtotal of \$700.00

When the discount tier is calculated

Then the applied discount tier should be "vip"

Copilot Chat

Agent

```
/bdd-steps feature=specs/discount-eligibility/discount-eligibility.feature stack=java
```

- Step definitions first, **first run fails**
- The table becomes data, not `ifs`
- Boundaries 9 vs 10, 499.99 vs 500.00 drive `>=`
- Highest `discount_percent` wins

✓ 8 scenarios pass

Spec, then plan, then code, and **review the plan** before any of it runs

 Copilot Chat

Agent

```
/sdd-plan spec=specs/password-reset/spec.md  
stack=node
```

spec.md



plan.md (stops)



tests



code

A strong plan

- Hashes tokens before storing them
- Injects the clock and the random source
- Maps every FR and NFR to a test

ASK THE ROOM

What would leak if the token store were dumped?

ASK THE ROOM

How do we test a 15-minute expiry without waiting 15 minutes?

Generate the docs, then **check them against the tests**

 Copilot Chat · LoyaltyService selected

/doc

 Then ask

Does this documentation match what the tests prove?

Why it matters

Docs that disagree with the tests are a bug report.
The tests are the executable truth; the docs should say the same thing.

Three habits to take back to **your team**



Scaffold

A spec plus repo instructions gives consistent, multi-layer code.



Debug

Evidence first. Root cause before fix. Then prove it with a test.



Test

Keep the loop small (TDD), make behavior the truth (BDD), plan before code (SDD).

Two stacks, one contract: **Copilot follows your conventions, not its favorite framework.**

Every module has a **known-good checkpoint**

STUCK IN...	DEMO: JUMP TO STAGE
Module 6	01-scaffold
Module 7	02-debug
Module 8	03-tests
Broken build demo	Demo: restore the build

Then restart the API. Java has no hot reload.

PROBLEM	FIX
Badge says offline	Only one backend can use port 3001
Port 3001 in use	<code>lsof -ti :3001 xargs kill</code>
Copilot quotes answers	Close <code>solutions/</code> tabs; keep <code>search.exclude</code>
Empty <code>#terminalLastCommand</code>	Use a regular terminal



Thank you

Build it. Break it. Prove it.

 docs.github.com/copilot

 Questions?